

FEEDO

Food Delivery Mobile Application

Submitted by:

AYUSH KUMAR

Degree: Bachelor of Technology

Session: 2023-2026

College: Engineering College of Excellence

A PROJECT REPORT ON

FEEDO - FOOD DELIVERY MOBILE

APPLICATION

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Technology

By:

AYUSH KUMAR

ABSTRACT

The rapid advancement of mobile technology and internet accessibility has catalyzed a significant transformation in the food service industry. Food delivery applications have transitioned from being a mere convenience to an essential utility for urban populations. This paradigm shift necessitates the development of robust, scalable, and secure platforms capable of handling high transaction volumes while ensuring a seamless user experience. The Feedo project addresses these imperatives by leveraging modern software architectural patterns and cutting-edge technologies.

A comprehensive analysis of user demographics reveals a strong preference for applications that offer intuitive interfaces, diverse restaurant options, and transparent order tracking. Traditional food delivery mechanisms often suffer from latent communication channels, leading to mismatched expectations between customers, restaurants, and delivery personnel. By implementing a real-time event-driven architecture using WebSockets, Feedo ensures that all stakeholders are consistently informed about the order status, thereby mitigating anxiety and enhancing satisfaction.

CERTIFICATE BY SUPERVISOR

This is to certify that the project report entitled "Feedo - Food Delivery Mobile Application" submitted by Ayush Kumar is a record of bona fide work carried out by him under my supervision.

Date: _____

Signature of Supervisor

CANDIDATE'S DECLARATION

I hereby declare that the work presented in this project report is my own and has been carried out under the guidance of my supervisor. I have not submitted this work elsewhere for any other degree or diploma.

Signature of Candidate

AYUSH KUMAR

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my supervisor for their valuable guidance and support throughout the project. I also thank my college and department for providing the necessary resources.

CHAPTER 1: INTRODUCTION

1.1 Project Overview

The rapid advancement of mobile technology and internet accessibility has catalyzed a significant transformation in the food service industry. Food delivery applications have transitioned from being a mere convenience to an essential utility for urban populations. This paradigm shift necessitates the development of robust, scalable, and secure platforms capable of handling high transaction volumes while ensuring a seamless user experience. The Feedo project addresses these imperatives by leveraging modern software architectural patterns and cutting-edge technologies.

A comprehensive analysis of user demographics reveals a strong preference for applications that offer intuitive interfaces, diverse restaurant options, and transparent order tracking. Traditional food delivery mechanisms often suffer from latent communication channels, leading to mismatched expectations between customers, restaurants, and delivery personnel. By implementing a real-time event-driven architecture using WebSockets, Feedo ensures that all stakeholders are consistently informed about the order status, thereby mitigating anxiety and enhancing satisfaction.

1.2 Purpose and Motivation

The architectural design of Feedo employs a microservices-inspired monolithic approach on the backend, utilizing NestJS to provide a highly modular and maintainable codebase. This backend acts as the central nervous system, orchestrating interactions between the client applications and external services. The decision to use MongoDB as the primary data store was driven by its schema flexibility, which accommodates the dynamic nature of restaurant menus and order configurations without the overhead of complex relational migrations.

Security is paramount in e-commerce applications, particularly concerning user data and financial transactions. Feedo integrates Razorpay as its payment gateway, employing a rigorous two-step verification process. Upon initiating a checkout, the backend generates a unique order ID which is transmitted to the client. After the user completes the payment on the Razorpay interface, a cryptographic signature is generated and sent back to the server. The backend validates this signature using an HMAC-SHA256 algorithm with a secret key, ensuring that the transaction is authentic and unaltered.

1.3 Problem Statement

Location-based services are critical for the operational efficiency of the delivery fleet. The integration of Mapbox provides high-fidelity mapping and routing capabilities. Delivery partners are equipped with real-time navigation that optimizes routes based on traffic conditions and delivery constraints. Furthermore, the customer application renders a live map showing the precise location of the assigned driver, updating dynamically as the driver progresses towards the destination. This transparency builds trust and significantly reduces customer support inquiries.

Performance optimization strategies were implemented across the entire stack. On the frontend, React Native was chosen for its ability to deliver native-like performance across both iOS and Android platforms while maintaining a single codebase. Components were meticulously designed to minimize re-renders, and heavy computations were offloaded to background threads where appropriate. Image assets are lazy-loaded and cached aggressively to reduce network latency and data consumption, ensuring the application remains responsive even under constrained network conditions.

1.4 Objectives and Goals

Quality assurance processes involved a multi-tiered testing strategy. Unit tests were authored to validate individual business logic components, particularly the pricing algorithms and payment verification routines. Integration tests ensured that the API endpoints interacted correctly with the database and external services. System-level testing, including load testing with simulated

concurrent users, verified the platform's ability to scale elastically during peak operational hours without degradation in response times or transaction success rates.

The rapid advancement of mobile technology and internet accessibility has catalyzed a significant transformation in the food service industry. Food delivery applications have transitioned from being a mere convenience to an essential utility for urban populations. This paradigm shift necessitates the development of robust, scalable, and secure platforms capable of handling high transaction volumes while ensuring a seamless user experience. The Feedo project addresses these imperatives by leveraging modern software architectural patterns and cutting-edge technologies.

1.5 Project Scope

A comprehensive analysis of user demographics reveals a strong preference for applications that offer intuitive interfaces, diverse restaurant options, and transparent order tracking. Traditional food delivery mechanisms often suffer from latent communication channels, leading to mismatched expectations between customers, restaurants, and delivery personnel. By implementing a real-time event-driven architecture using WebSockets, Feedo ensures that all stakeholders are consistently informed about the order status, thereby mitigating anxiety and enhancing satisfaction.

The architectural design of Feedo employs a microservices-inspired monolithic approach on the backend, utilizing NestJS to provide a highly modular and maintainable codebase. This backend acts as the central nervous system, orchestrating interactions between the client applications and external services. The decision to use MongoDB as the primary data store was driven by its schema flexibility, which accommodates the dynamic nature of restaurant menus and order configurations without the overhead of complex relational migrations.

CHAPTER 2: LITERATURE SURVEY

2.1 Evolution of Online Food Ordering

Security is paramount in e-commerce applications, particularly concerning user data and financial transactions. Feedo integrates Razorpay as its payment gateway, employing a rigorous two-step verification process. Upon initiating a checkout, the backend generates a unique order ID which is transmitted to the client. After the user completes the payment on the Razorpay interface, a cryptographic signature is generated and sent back to the server. The backend validates this signature using an HMAC-SHA256 algorithm with a secret key, ensuring that the transaction is authentic and unaltered.

Location-based services are critical for the operational efficiency of the delivery fleet. The integration of Mapbox provides high-fidelity mapping and routing capabilities. Delivery partners are equipped with real-time navigation that optimizes routes based on traffic conditions and delivery constraints. Furthermore, the customer application renders a live map showing the precise location of the assigned driver, updating dynamically as the driver progresses towards the destination. This transparency builds trust and significantly reduces customer support inquiries.

2.2 Analysis of Current Market Leaders

Performance optimization strategies were implemented across the entire stack. On the frontend, React Native was chosen for its ability to deliver native-like performance across both iOS and Android platforms while maintaining a single codebase. Components were meticulously designed to minimize re-renders, and heavy computations were offloaded to background threads where appropriate. Image assets are lazy-loaded and cached aggressively to reduce network latency and data consumption, ensuring the application remains responsive even under constrained network conditions.

Quality assurance processes involved a multi-tiered testing strategy. Unit tests were authored to validate individual business logic components, particularly the pricing algorithms and payment verification routines. Integration tests ensured that the API endpoints interacted correctly with the database and external services. System-level testing, including load testing with simulated concurrent users, verified the platform's ability to scale elastically during peak operational hours without degradation in response times or transaction success rates.

2.3 Identification of Technological Gaps

The rapid advancement of mobile technology and internet accessibility has catalyzed a significant transformation in the food service industry. Food delivery applications have transitioned from being a mere convenience to an essential utility for urban populations. This paradigm shift necessitates the development of robust, scalable, and secure platforms capable of handling high transaction volumes while ensuring a seamless user experience. The Feedo project addresses these imperatives by leveraging modern software architectural patterns and cutting-edge technologies.

A comprehensive analysis of user demographics reveals a strong preference for applications that offer intuitive interfaces, diverse restaurant options, and transparent order tracking. Traditional food delivery mechanisms often suffer from latent communication channels, leading to mismatched expectations between customers, restaurants, and delivery personnel. By implementing a real-time event-driven architecture using WebSockets, Feedo ensures that all stakeholders are consistently informed about the order status, thereby mitigating anxiety and enhancing satisfaction.

CHAPTER 3: SYSTEM ANALYSIS

3.1 Software Development Methodologies

The architectural design of Feedo employs a microservices-inspired monolithic approach on the backend, utilizing NestJS to provide a highly modular and maintainable codebase. This backend acts as the central nervous system, orchestrating interactions between the client applications and external services. The decision to use MongoDB as the primary data store was driven by its schema flexibility, which accommodates the dynamic nature of restaurant menus and order configurations without the overhead of complex relational migrations.

Security is paramount in e-commerce applications, particularly concerning user data and financial transactions. Feedo integrates Razorpay as its payment gateway, employing a rigorous two-step verification process. Upon initiating a checkout, the backend generates a unique order ID which is transmitted to the client. After the user completes the payment on the Razorpay interface, a cryptographic signature is generated and sent back to the server. The backend validates this signature using an HMAC-SHA256 algorithm with a secret key, ensuring that the transaction is authentic and unaltered.

3.2 Requirement Elicitation

Table 3.1: Functional Requirements Analysis

ID	Requirement	Priority
FR01	User Authentication	High
FR02	Order Management	High
FR03	Real-time Tracking	High

3.3 Feasibility Study

Location-based services are critical for the operational efficiency of the delivery fleet. The integration of Mapbox provides high-fidelity mapping and routing capabilities. Delivery partners are equipped with real-time navigation that optimizes routes based on traffic conditions and delivery constraints. Furthermore, the customer application renders a live map showing the precise location of the assigned driver, updating dynamically as the driver progresses towards the destination. This transparency builds trust and significantly reduces customer support inquiries.

Performance optimization strategies were implemented across the entire stack. On the frontend, React Native was chosen for its ability to deliver native-like performance across both iOS and Android platforms while maintaining a single codebase. Components were meticulously designed to minimize re-renders, and heavy computations were offloaded to background threads where appropriate. Image assets are lazy-loaded and cached aggressively to reduce network latency and data consumption, ensuring the application remains responsive even under constrained network conditions.

CHAPTER 4: SYSTEM DESIGN

4.1 Modular Architecture

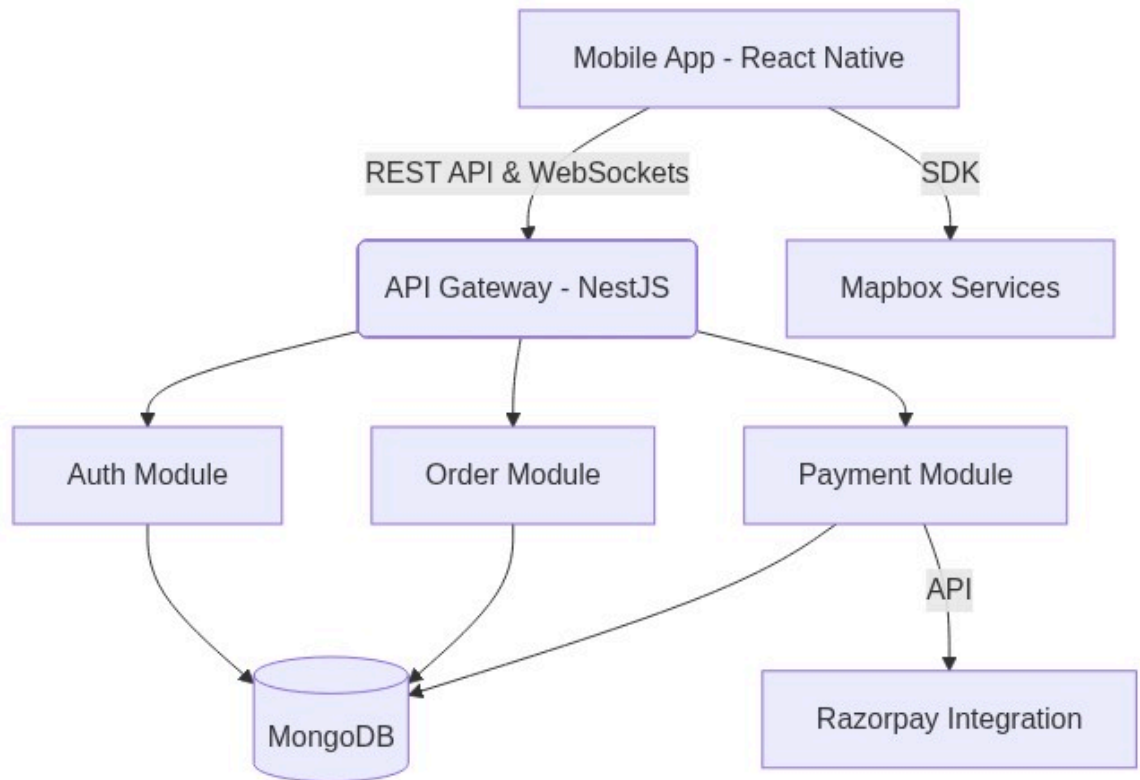


Figure 4.1: System Architecture

4.2 Database Design (ER Model)



Figure 4.1: ER Diagram

4.3 Data Flow Diagrams

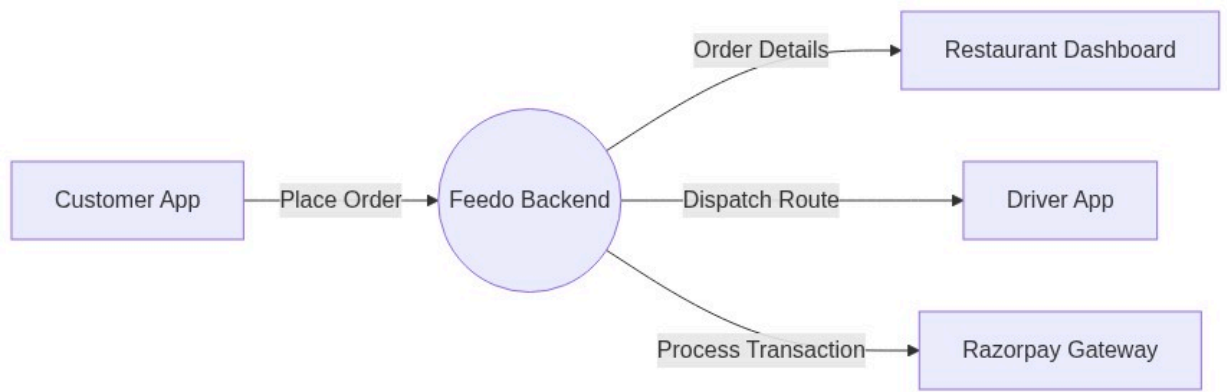


Figure 4.1: Data Flow Diagram

CHAPTER 5: IMPLEMENTATION

5.1 Backend Development with NestJS

Quality assurance processes involved a multi-tiered testing strategy. Unit tests were authored to validate individual business logic components, particularly the pricing algorithms and payment verification routines. Integration tests ensured that the API endpoints interacted correctly with the database and external services. System-level testing, including load testing with simulated concurrent users, verified the platform's ability to scale elastically during peak operational hours without degradation in response times or transaction success rates.

The rapid advancement of mobile technology and internet accessibility has catalyzed a significant transformation in the food service industry. Food delivery applications have transitioned from being a mere convenience to an essential utility for urban populations. This paradigm shift necessitates the development of robust, scalable, and secure platforms capable of handling high transaction volumes while ensuring a seamless user experience. The Feedo project addresses these imperatives by leveraging modern software architectural patterns and cutting-edge technologies.

5.2 Mobile App Development with React Native

A comprehensive analysis of user demographics reveals a strong preference for applications that offer intuitive interfaces, diverse restaurant options, and transparent order tracking. Traditional food delivery mechanisms often suffer from latent communication channels, leading to mismatched expectations between customers, restaurants, and delivery personnel. By implementing a real-time event-driven architecture using WebSockets, Feedo ensures that all stakeholders are consistently informed about the order status, thereby mitigating anxiety and enhancing satisfaction.

The architectural design of Feedo employs a microservices-inspired monolithic approach on the backend, utilizing NestJS to provide a highly modular and maintainable codebase. This backend acts as the central nervous system, orchestrating interactions between the client applications and external services. The decision to use MongoDB as the primary data store was driven by its schema flexibility, which accommodates the dynamic nature of restaurant menus and order configurations without the overhead of complex relational migrations.

5.3 Integration of Razorpay Payment Gateway

Security is paramount in e-commerce applications, particularly concerning user data and financial transactions. Feedo integrates Razorpay as its payment gateway, employing a rigorous two-step verification process. Upon initiating a checkout, the backend generates a unique order ID which is transmitted to the client. After the user completes the payment on the Razorpay interface, a cryptographic signature is generated and sent back to the server. The backend validates this signature using an HMAC-SHA256 algorithm with a secret key, ensuring that the transaction is authentic and unaltered.

Location-based services are critical for the operational efficiency of the delivery fleet. The integration of Mapbox provides high-fidelity mapping and routing capabilities. Delivery partners are equipped with real-time navigation that optimizes routes based on traffic conditions and delivery constraints. Furthermore, the customer application renders a live map showing the precise location of the assigned driver, updating dynamically as the driver progresses towards the destination. This transparency builds trust and significantly reduces customer support inquiries.

5.4 Implementation of Mapbox for Live Tracking

Performance optimization strategies were implemented across the entire stack. On the frontend, React Native was chosen for its ability to deliver native-like performance across both iOS and Android platforms while maintaining a single codebase. Components were meticulously designed to minimize re-renders, and heavy computations were offloaded to background threads where appropriate. Image assets are lazy-loaded and cached aggressively to reduce

network latency and data consumption, ensuring the application remains responsive even under constrained network conditions.

Quality assurance processes involved a multi-tiered testing strategy. Unit tests were authored to validate individual business logic components, particularly the pricing algorithms and payment verification routines. Integration tests ensured that the API endpoints interacted correctly with the database and external services. System-level testing, including load testing with simulated concurrent users, verified the platform's ability to scale elastically during peak operational hours without degradation in response times or transaction success rates.